MiniEVM 애플리케이션 구조 분석

1. MinitiaApp 구조체

`MinitiaApp` 은 전체 애플리케이션의 핵심 구조체로, Cosmos SDK의 `BaseApp` 을 확장합니다.

```
type MinitiaApp struct {
*baseapp.BaseApp
keepers.AppKeepers

// address codecs
ac, vc, cc address.Codec

legacyAmino *codec.LegacyAmino
appCodec codec.Codec
txConfig client.TxConfig
interfaceRegistry types.InterfaceRegistry

// the module manager
ModuleManager *module.Manager
BasicModuleManager module.BasicManager
```

```
// the configurator

configurator module.Configurator



// Override of BaseApp's CheckTx

checkTxHandler blockchecktx.CheckTx



// evm indexer

evmIndexer evmindexer.EVMIndexer



// checktx wrapper

checkTxWrapper *checktx.CheckTxWrapper



// post handler for tracing

postHandler sdk.PostHandler

}
```

2. 애플리케이션 초기화 과정

2.1 NewMinitiaApp 함수 분석

```
func  NewMinitiaApp(

logger log.Logger,

db dbm.DB,
```

```
indexerDB dbm.DB,
traceStore io.Writer,
loadLatest  bool,
evmConfig evmconfig.EVMConfig,
appOpts servertypes.AppOptions,
baseAppOptions ...func(*baseapp.BaseApp),
) *MinitiaApp
```

초기화 단계:

1. **설정 로드**

```
mempoolMaxTxs := cast.ToInt(appOpts.Get(server.FlagMempoolMaxTxs))
queryGasLimit := cast.ToInt(appOpts.Get(server.FlagQueryGasLimit))
```

2. **인코딩 설정**

```
encodingConfig := params.MakeEncodingConfig()
std.RegisterLegacyAminoCodec(encodingConfig.Amino)
std.RegisterInterfaces(encodingConfig.InterfaceRegistry)
cryptocodec.RegisterLegacyAminoCodec(encodingConfig.Amino)
cryptocodec.RegisterInterfaces(encodingConfig.InterfaceRegistry)
```

3. **BaseApp 생성**

```
bApp := baseapp.NewBaseApp(AppName, logger, db, encodingConfig.TxConfig.TxDecode
```

```
bApp.SetCommitMultiStoreTracer(traceStore)

bApp.SetVersion(version.Version)

bApp.SetInterfaceRegistry(interfaceRegistry)

bApp.SetTxEncoder(txConfig.TxEncoder())
```

3. Keepers 관리

3.1 AppKeepers 구조

모든 모듈의 Keeper들을 중앙 집중식으로 관리:

```
app.AppKeepers = keepers.NewAppKeeper(

app.ac, app.vc, app.cc,

appCodec,

txConfig,

bApp,

legacyAmino,

maccPerms,

blockedModuleAccountAddrs,

skipUpgradeHeights,

homePath,

invCheckPeriod,

logger,

evmConfig,
```

```
appOpts,
)
```

3.2 주요 Keeper들

- **AccountKeeper**: 계정 관리
- **BankKeeper**: 토큰 전송 및 잔액 관리
- **EVMKeeper**: EVM 상태 및 실행 관리
- **IBCKeeper**: Inter-Blockchain Communication
- **StakingKeeper**: 스테이킹 관련 기능

4. 모듈 관리자

4.1 ModuleManager 설정

```
app.ModuleManager = module.NewManager(appModules(app, skipGenesisInvariants)...)
```

4.2 모듈 실행 순서 설정

PreBlockers

```
app.ModuleManager.SetOrderPreBlockers(
upgradetypes.ModuleName,
evmtypes.ModuleName,
)
```

BeginBlockers & EndBlockers

```
app.ModuleManager.SetOrderBeginBlockers(orderBeginBlockers()...)

app.ModuleManager.SetOrderEndBlockers(orderEndBlockers()...)
```

Genesis 순서

```
genesisModuleOrder := orderInitBlockers()

app.ModuleManager.SetOrderInitGenesis(genesisModuleOrder...)

app.ModuleManager.SetOrderExportGenesis(genesisModuleOrder...)
```

5. 인덱서 설정

5.1 EVM 인덱서 초기화

```
evmIndexer, streamingManager, err := setupIndexer(app, indexerDB)

if  err != nil {

tmos.Exit(err.Error())

}


app.SetEVMIndexer(evmIndexer)

app.SetStreamingManager(*streamingManager)
```

5.2 인덱서 역할

- 블록체인 데이터 실시간 인덱싱
- JSON-RPC 쿼리 성능 최적화
- 이벤트 로그 검색 지원

6. BlockSDK 통합

6.1 BlockSDK 설정

```
mempool, anteHandler, checkTx, prepareProposalHandler, processProposalHandler, e
```

6.2 핸들러 오버라이드

```
app.SetMempool(mempool)

app.SetAnteHandler(anteHandler)

app.SetPrepareProposal(prepareProposalHandler)

app.SetProcessProposal(processProposalHandler)

app.SetCheckTx(checkTx)
```

7. ABCI 메서드 구현

7.1 PreBlocker

```
func (app *MinitiaApp) PreBlocker(ctx sdk.Context, _ *abci.RequestFinalizeBlock)

return  app.ModuleManager.PreBlock(ctx)
```

```
}
```

7.2 BeginBlocker

```
func (app *MinitiaApp) BeginBlocker(ctx sdk.Context) (sdk.BeginBlock, error) {
return  app.ModuleManager.BeginBlock(ctx)
}
```

7.3 EndBlocker

```
func (app *MinitiaApp) EndBlocker(ctx sdk.Context) (sdk.EndBlock, error) {
return  app.ModuleManager.EndBlock(ctx)
}
```

7.4 InitChainer

```
func (app *MinitiaApp) InitChainer(ctx sdk.Context, req *abci.RequestInitChain)
var  genesisState GenesisState
if  err := tmjson.Unmarshal(req.AppStateBytes, &genesisState);  err != nil {
tmos.Exit(err.Error())
}
if  err := app.UpgradeKeeper.SetModuleVersionMap(ctx, app.ModuleManager.GetVersi
tmos.Exit(err.Error())
```

```
}

return  app.ModuleManager.InitGenesis(ctx, app.appCodec, genesisState)

}
```

8. PostHandler 설정

8.1 PostHandler 초기화

```
func (app *MinitiaApp) setPostHandler() {

app.postHandler = posthandler.NewPostHandler(

app.Logger(),

app.EVMKeeper,

)


app.SetPostHandler(app.postHandler)

}
```

8.2 PostHandler 역할

- 트랜잭션 실행 후 추가 처리
- EVM 트레이싱 및 로깅
- 이벤트 발생 및 인덱싱 트리거

9. 스냅샷 확장

9.1 스냅샷 확장 등록

```
if manager := app.SnapshotManager(); manager != nil && app.evmIndexer != nil {
err := manager.RegisterExtensions(
app.evmIndexer,
)
if err != nil {
panic(fmt.Errorf("failed to register snapshot extension: %s", err))
}
}
```

9.2 스냅샷 기능

- 상태 스냅샷 생성 및 복원
- 빠른 동기화 지원
- 인덱서 데이터 포함

10. 애플리케이션 생명주기

10.1 시작 과정

1. 설정 로드 및 검증
2. 데이터베이스 연결
3. Keeper 초기화
4. 모듈 등록 및 설정

5. 인덱서 시작

6. 네트워크 서비스 시작

10.2 종료 과정

```
func (app *MinitiaApp) Close() error {
var  errs []error

if  err := app.BaseApp.Close();  err != nil {
errs = append(errs, err)
}

if  app.evmIndexer != nil {
if  err := app.evmIndexer.Close();  err != nil {
errs = append(errs, err)
}
}

if  app.checkTxWrapper != nil {
app.checkTxWrapper.Stop()
}

return  errors.Join(errs...)
```

```
}
```

이 구조를 통해 MiniEVM은 Cosmos SDK의 모든 기능을 활용하면서도 EVM 호환성을 제공하는 통합된 블록체인 애플리케이션을 구현합니다.